# RELIEFLINK - REAL-TIME DISASTER RELIEF COORDINATION PLATFORM

**(RELIEFLINK)**

**A PROJECT REPORT**

*Submitted by*

**Sanjay K (310624104052)**

*in partial fulfillment for the award of the Degree*
*of*

**BACHELOR OF ENGINEERING**

*in*

**COMPUTER SCIENCE AND ENGINEERING**

**EASWARI ENGINEERING COLLEGE,**
**(AN AUTONOMOUS INSTITUTION)**
**AFFILIATED TO ANNA UNIVERSITY, CHENNAI**

**OCTOBER 2026**

# EASWARI ENGINEERING COLLEGE

## BONAFIDE CERTIFICATE

Certified that this mini project report titled “RELIEFLINK - REAL-TIME DISASTER RELIEF COORDINATION PLATFORM” is the Bonafide work of “SANJAY K (310624104052)”, who carried out the project work under my supervision.

**HEAD OF THE DEPARTMENT**

**Dr. S. Cloudin**
Easwari Engineering College
Ramapuram, Chennai - 600089

**SUPERVISOR**

**Mrs. Nagamani Segalla**
Easwari Engineering College
Ramapuram, Chennai - 600089

# EASWARI ENGINEERING COLLEGE

## Certificate of Viva-Voce Examination

This is to certify that the Viva Voce Examination of the Project Work entitled “ReliefLink: Real-Time Disaster Relief Coordination Platform” submitted by Mr. Sanjay K (310624104052) in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering was conducted on 27-10-2026 in the Department of Computer Science and Engineering, Easwari Engineering College, Ramapuram – Chennai.

**Internal Examiner**

Date: 27-10-2026

# ACKNOWLEDGEMENT

I express my sincere gratitude to the Principal Dr. P. Deiva Sundari and the Management of our college for providing me with the opportunity to carry out this project work.

I convey my heartfelt thanks to the Head of the Department, Dr. S. Cloudin, Department of Computer Science and Engineering, for the continuous encouragement and support.

I am extremely thankful to my project guide Mrs. Nagamani Segalla, Department of Computer Science and Engineering, for valuable guidance, suggestions and motivation throughout the completion of this project.

I would also like to extend my gratitude to all the faculty members of the Department of Computer Science and Engineering for their support.

Finally, I thank my parents and friends for their constant encouragement and help during the course of this project.

# TABLE OF CONTENTS


| S. No. | Contents | Page No. |
|---|---|---|
| 1 | Abstract | i |
| 2 | List of Figures | ii |
| 3 | List of Tables | iii |
| 4 | List of Abbreviations | iv |
| 5 | Chapter 1 – Introduction | 1 |
| 5.1 | Project Overview | 1 |
| 5.2 | Problem Statement | 2 |
| 5.3 | Objectives of the Project | 2 |
| 5.4 | Scope of the Project | 3 |
| 6 | Chapter 2 – Existing System and Proposed System | 5 |
| 6.1 | Existing System | 5 |
| 6.2 | Limitations of the Existing System | 5 |
| 6.3 | Proposed Solution | 5 |
| 6.4 | Advantages of the Proposed System | 6 |
| 7 | Chapter 3 – System Analysis and Design | 8 |
| 7.1 | Hardware and Software Requirements | 8 |
| 7.2 | System Architecture | 9 |
| 7.3 | Module Description | 11 |
| 7.4 | UML Diagrams / Data Flow Diagram | 13 |
| 7.5 | Algorithm / Flowchart | 17 |
| 8 | Chapter 4 – Implementation | 21 |
| 8.1 | Implementation Details | 21 |
| 8.2 | Module Implementation | 22 |
| 8.3 | Tools and Technologies Used | 24 |
| 9 | Chapter 5 – Results and Testing | 26 |
| 9.1 | Screenshots and Outputs | 26 |
| 9.2 | Testing and Validation | 29 |
| 9.3 | Results and Discussion | 31 |
| 10 | Chapter 6 – Conclusion and Future Enhancement | 33 |
| 10.1 | Conclusion | 33 |
| 10.2 | Future Enhancements | 33 |
| 11 | References / Bibliography | 35 |
| 12 | Appendices (Source Code) | 36 |

# ABSTRACT

ReliefLink is a centralised, production-grade real-time disaster relief coordination platform engineered to unify distressed citizens, accredited on-ground volunteers, civil society organisations, and government disaster management authorities (NDRF/SDMA) across India. It is built on a resilient modern full-stack architecture comprising React 19 with Tailwind CSS and Leaflet GIS on the front end, backed by Node.js, Express, Socket.IO, and an embedded PostgreSQL (PGlite) engine for transactional persistence. By structuring every emergency dispatch into real-time geospatial distress feeds, the platform operates without proprietary hardware dependencies while bridging the life-critical gap between distress reporting and resource delivery.

The project addresses three catastrophic bottlenecks inherent to traditional disaster management in India: chaotic, unindexed distress reporting via fragmented social media and overloaded telephone helplines; severe absence of geofenced tactical coordination leading to duplicated volunteer missions and neglected high-casualty pockets; and zero automated SLA triage monitoring resulting in unescalated life-threatening emergencies. To solve them, ReliefLink introduces one-tap GPS-assisted SOS reporting with category-specific triage scoring (Medical Emergency, Trapped Civilians, Flood Evacuation, Food & Potable Water, Structural Collapse). An autonomous background daemon evaluates SLA compliance in real time, automatically escalating neglected P0/P1 incidents to senior command coordinators when triage thresholds are breached.

This report presents the system analysis, architectural blueprint, engineering implementation, and comprehensive testing validation of ReliefLink. Real-time synchronisation is governed by bi-directional WebSocket channels, enabling incident commanders to monitor live tactical maps, verify distress veracity, dispatch verified volunteer squads, track emergency shelter capacities, and export geocoded situation audit logs in CSV format.

**Keywords:** disaster relief coordination, real-time telemetry, WebSocket, Leaflet GIS, SLA escalation engine, PGlite/PostgreSQL, triage matrix, emergency shelter tracking.

# LIST OF FIGURES


| FIGURE NO. | TITLE | PAGE NO. |
|---|---|---|
| 3.1 | System architecture | 10 |
| 3.2 | Use case diagram | 14 |
| 3.3 | Entity relationship diagram | 15 |
| 3.4 | Sequence diagram of distress SOS ingestion and dispatch | 16 |
| 3.5 | Level 1 data flow diagram | 17 |
| 3.6 | Flowchart of SOS submission and geo-verification | 18 |
| 3.7 | Flowchart of SLA escalation and automated priority worker | 19 |
| 5.1 | Live tactical command dashboard with GIS map | 26 |
| 5.2 | SOS distress queue and verification panel | 26 |
| 5.3 | Citizen SOS distress reporting modal with GPS lock | 27 |
| 5.4 | Real-time casualty trends and triage distribution graphs | 27 |
| 5.5 | Volunteer task force dispatch and mission management | 28 |
| 5.6 | Emergency shelter capacity and resource monitor | 28 |
| 5.7 | Automated SLA escalation alerts and triage timer panel | 29 |
| 5.8 | Disaster incident audit report and CSV data export | 29 |

# LIST OF TABLES


| TABLE NO. | TITLE | PAGE NO. |
|---|---|---|
| 1.1 | ReliefLink at a glance | 1 |
| 1.2 | Objectives and how they are met in ReliefLink | 3 |
| 2.1 | Comparison of the existing process and ReliefLink | 6 |
| 3.1 | Functional requirements | 8 |
| 3.2 | Software requirements | 9 |
| 3.3 | Minimum hardware requirements | 9 |
| 3.4 | Layers of the system | 10 |
| 3.5 | Proposed REST API routes | 10 |
| 3.6 | Disaster incident and SOS validation bounds | 11 |
| 3.7 | Main pages and routes | 12 |
| 3.8 | Summary of modules | 13 |
| 3.9 | Data requirements | 13 |
| 3.10 | Priority matrix and SLA escalation rules | 13 |
| 3.11 | Entity summary | 15 |
| 4.1 | Reusable interface components | 22 |
| 4.2 | Libraries, frameworks and services | 24 |
| 4.3 | Development tools | 25 |
| 5.1 | Test cases and results | 30 |
| 5.2 | Test coverage by area | 31 |
| 6.1 | Roadmap of future work | 34 |

# LIST OF ABBREVIATIONS

| ABBREVIATION | EXPANSION |
|---|---|
| API | Application Programming Interface |
| CRUD | Create, Read, Update, Delete |
| CSS | Cascading Style Sheets |
| CSV | Comma-Separated Values |
| DFD | Data Flow Diagram |
| ER | Entity Relationship |
| GIS | Geographic Information System |
| GPS | Global Positioning System |
| HTTP | Hypertext Transfer Protocol |
| HTTPS | Hypertext Transfer Protocol Secure |
| JSON | JavaScript Object Notation |
| JWT | JSON Web Token |
| NDRF | National Disaster Response Force (India) |
| PGlite | PostgreSQL in WebAssembly / Embedded Node.js Engine |
| PWA | Progressive Web Application |
| REST | Representational State Transfer |
| SDMA | State Disaster Management Authority |
| SLA | Service Level Agreement |
| SOS | Save Our Souls (Emergency Distress Signal) |
| UI | User Interface |
| UML | Unified Modeling Language |
| W3C | World Wide Web Consortium |

# CHAPTER 1
# INTRODUCTION

## 1.1 Project Overview

ReliefLink is a centralised, full-stack disaster relief coordination platform engineered to address the critical communication breakdowns that occur during climate catastrophes, extreme flooding, cyclones, and humanitarian emergencies across India. During major natural calamities—such as the Chennai floods, Wayanad landslides, or Odisha cyclones—disaster response mechanisms face acute challenges. Distress messages arrive in chaotic fragments across WhatsApp groups, social feeds, and saturated 112/1070 emergency call centres. ReliefLink replaces these disjointed channels with a unified, high-reliability command workspace connecting affected citizens, accredited field volunteers, civil defense teams, and administrative disaster authorities.

The application is architected as an end-to-end full-stack web system. The client interface is built using React 19, styled with Tailwind CSS, and powered by an interactive Leaflet GIS tactical mapping engine with high-contrast emergency visibility. The backend consists of a Node.js and Express HTTP server integrated with Socket.IO for sub-second real-time event broadcasting. Persistent data storage is governed by a fully embedded PostgreSQL engine (PGlite), guaranteeing strict relational schema integrity, transaction safety, and comprehensive auditability without external database hosting overheads.

Three core design innovations distinguish ReliefLink from generic emergency portals:

- **Sub-Second Geospatial SOS Ingestion.** Citizens trigger emergency distress requests with automatic GPS coordinate locking, category tagging (Medical, Trapped, Food/Water, Shelter, Rescue), and casualty counts without requiring tedious account setup.
- **Automated SLA Escalation Engine.** An autonomous background worker runs continuously on the server, tracking response delays against rigorous Indian National Disaster Management Authority (NDMA) triage standards and automatically escalating unassigned or neglected distress beacons.
- **Synchronous Tactical Dispatch & Shelter Registry.** Incident commanders dispatch verified volunteer squads to live distress pins and monitor real-time shelter bed and ration capacities with immediate WebSocket propagation.

**Table 1.1 ReliefLink at a glance**

| Item | Detail |
|---|---|
| Project name | ReliefLink - Real-Time Disaster Relief Coordination Platform |
| Type | Full-stack web application with real-time WebSocket telemetry and background worker |
| Architecture | 100% web software; client-server with Leaflet GIS and PGlite embedded PostgreSQL |
| Users | Affected Citizens, On-Ground Volunteers, Disaster Command Admins (NDRF/SDMA) |
| Front end | React 19, TypeScript, Tailwind CSS, Leaflet OpenStreetMap, Lucide Icons, Motion |
| Back end | Node.js, Express.js (REST APIs), Socket.IO (WebSockets), SLA Escalation Daemon |
| Database | PostgreSQL (via embedded @electric-sql/pglite) with relational integrity and migrations |
| Geocoding | W3C Browser Geolocation API + OpenStreetMap Nominatim reverse-geocoding |
| Cadence | Sub-second real-time broadcasts; 60-second SLA escalation evaluation cycle |
| Course | Full Stack Development (B.E. Computer Science and Engineering) |

## 1.2 Problem Statement

During rapid-onset environmental emergencies and natural disasters in India, relief efforts suffer from four critical operational bottlenecks that severely impair life-saving interventions:

- **1. Chaotic & Duplicated Distress Feeds.** Victims post desperate distress messages across multiple social media platforms and message boards without standardized coordinates. Relief agencies waste precious hours verifying duplicates while isolated victims remain unnoticed.
- **2. Absence of Geospatial Tactical Triage.** Disaster management helplines operate via verbal phone calls and static spreadsheets, lacking real-time visual mapping of flood levels, road blockages, and casualty concentrations.
- **3. Uncoordinated Volunteer Deployment.** Good Samaritan volunteers and NGOs deploy haphazardly to easily accessible urban centres, leaving remote or severely submerged rural hamlets without medical personnel, drinking water, or rescue boats.
- **4. Zero Automated SLA Triage Tracking.** Without systematic priority clocks, critical life-or-death SOS beacons (e.g., stranded infants or critical oxygen shortages) sit in the same queue as minor supply requests, resulting in preventable loss of life.

## 1.3 Objectives of the Project

The primary objective of ReliefLink is to provide a reliable, zero-latency digital backbone for disaster distress management. The specific objectives and their corresponding engineering implementations are summarised in Table 1.2.

- **Provide an instant-access, zero-barrier SOS distress portal** enabling stranded citizens to report emergencies with precise GPS geolocation and triage categorization.
- **Deliver a real-time tactical command GIS map** providing incident commanders with live situation awareness, cluster heatmaps, and danger zones.
- **Implement an automated SLA escalation daemon** that enforces priority response windows and automatically escalates stagnant high-severity distress requests.
- **Establish a structured volunteer task force management module** for credential verification, task assignment, status updates, and field auditability.
- **Create an emergency shelter and supply inventory tracking system** preventing overcrowding and directing displaced citizens to verified relief camps.

**Table 1.2 Objectives and how they are met in ReliefLink**

| Objective | How it is met in ReliefLink |
|---|---|
| Centralised SOS Distress Registry | Module 1 - Instant SOS modal with GPS reverse-geocoding, priority triage tag, and deduplication |
| Real-Time Tactical GIS Command | Module 2 - Leaflet interactive live map with real-time Socket.IO pin updates and hazard clustering |
| Volunteer & Mission Coordination | Module 3 - Volunteer task force portal with skill-based mission assignment and status transitions |
| Emergency Shelter & Logistics | Module 3 - Live relief camp registry with occupancy counters, meal supplies, and medical inventory |
| Automated SLA Escalation Engine | Module 4 - Node.js background worker evaluating priority deadlines (P0/P1/P2/P3) every 60 seconds |
| Situation Auditing & Analytics | Module 4 - Visual incident metrics, severity breakdown charts, and instantaneous CSV report exports |

## 1.4 Scope of the Project

The project encompasses the complete web-based software ecosystem supporting disaster incident management: citizen distress intake, real-time command dispatch, volunteer tasking, shelter management, and administrative auditing.

**Inside the scope**

- **Citizen SOS Reporting:** One-tap distress submission with browser GPS acquisition, reverse-geocoded location, severity grading (P0 to P3), and contact info.
- **Tactical GIS Command Map:** Interactive map rendering live distress pins, volunteer squad locations, and relief shelters with colour-coded priority rings.
- **Real-Time Bi-Directional Telemetry:** Instantaneous propagation of new emergencies, status updates, and assignments to all connected clients via Socket.IO.
- **Automated SLA Escalation Engine:** Server-side background worker monitoring response countdowns and elevating unaddressed high-priority distress calls.
- **Volunteer Task Force Lifecycle:** Registration, team assignment, task acceptance, on-scene verification, and rescue completion.
- **Shelter & Relief Camp Logistics:** Live tracking of shelter capacity, current occupancy, drinking water, medical kits, and baby food availability.
- **Data Export & Audit Logs:** Comprehensive situation audit logs with geocoded CSV streaming for NDMA/ SDMA administrative reporting.

**Outside the scope of this version**

- **Satellite Uplink Hardware:** The current prototype operates over cellular/broadband IP networks; dedicated satellite modems are earmarked for future iterations.
- **Automated Drone Flight Routing:** While landing zone coordinates are recorded, physical autopilot telemetry is managed externally.
- **SMS Gateway Telephony:** Distress broadcasts are handled in-app via WebSockets; native cellular SMS fallback is scheduled for Phase 2.

## Organisation of the report

The remainder of this report is organised as follows: Chapter 2 discusses the existing manual mechanisms and presents the proposed ReliefLink platform architecture. Chapter 3 provides detailed requirements analysis, system architecture, database schema, UML diagrams, and algorithmic models. Chapter 4 details the software engineering implementation, modular components, and toolchains. Chapter 5 evaluates experimental testing outcomes with 22 test cases and screen outputs. Chapter 6 concludes the report with key milestones and future roadmaps, followed by academic references and complete technical appendices.

# CHAPTER 2
# EXISTING SYSTEM AND PROPOSED SYSTEM

## 2.1 Existing System

In traditional disaster management operations across India, emergency response relies predominantly on manual, decentralised, and fragmented procedures:

- **Emergency Helplines (112 / 1070):** Operators manually take down verbal descriptions of landmarks over copper or cellular telephone lines, creating transcription errors and losing exact GPS accuracy.
- **Ad-Hoc Social Media Amplification:** Distressed families post pleas on X (Twitter), Facebook, and WhatsApp groups. These posts are repeatedly retweeted and forwarded for days after the victims have already been rescued, flooding communication channels with obsolete information.
- **Paper Logs and Disconnected Spreadsheets:** Control rooms record distress entries in paper ledgers or standalone spreadsheets. Volunteer squads receive verbal instructions, leaving no digital audit trail of who was dispatched or when rescue was achieved.
- **Unverified Shelter Capacities:** Displaced citizens walk miles towards flood relief camps only to find them overflowing, lacking food supplies, or shut down.

## 2.2 Limitations of the Existing System

- **1. Massive Data Redundancy:** No automated deduplication mechanism exists, resulting in multiple teams dispatching to the same household while adjacent streets are neglected.
- **2. Absence of GPS Precision:** Descriptive text such as "near the old banyan tree behind the temple" becomes useless when floodwaters submerge all physical landmarks.
- **3. Critical Response Delays:** Without automated timers, life-threatening calls (pregnant women, medical trauma) wait in the same chronological queue as general dry-ration requests.
- **4. Volunteer Safety Hazards:** Volunteer squads enter flash-flood or landslide zones without real-time situational awareness or status check-ins.
- **5. Zero Post-Disaster Auditability:** Government agencies cannot accurately audit relief distribution, identify operational bottlenecks, or account for missing persons.

## 2.3 Proposed Solution

ReliefLink consolidates all relief workflows into a unified, high-performance web platform. Stranded citizens submit distress requests in two taps with exact GPS telemetry. Control rooms monitor a real-time tactical Leaflet map. Volunteer task forces receive verified assignments with navigation coordinates, and shelters broadcast live bed and meal availability. A background SLA escalation worker ensures zero high-priority calls remain unanswered.

**Table 2.1 Comparison of the existing process and ReliefLink**

| Aspect | Existing Manual Process | ReliefLink Platform |
|---|---|---|
| Distress Intake | Verbal phone calls (112) and chaotic social media posts | Instant web SOS with browser GPS lock and category triage |
| Geospatial Visibility | Static paper maps and vague landmark descriptions | Dynamic Leaflet GIS map with live color-coded priority markers |
| Data Synchronisation | Manual phone trees and periodic spreadsheet consolidation | Sub-second real-time bi-directional WebSocket broadcast (Socket.IO) |
| Triage & Prioritisation | First-come-first-served chronological queues | Dynamic severity scoring (P0 Critical, P1 High, P2 Moderate, P3 Low) |
| SLA Monitoring | Manual memory; neglected cases frequently forgotten | Autonomous daemon evaluating SLAs every 60s with auto-escalation |
| Volunteer Dispatch | Ad-hoc groups deploying without coordination | Role-based task force portal with skill matching and mission tracking |
| Shelter Logistics | Unchecked word-of-mouth reports | Live occupancy meters, meal counts, and medical supply registers |
| Post-Disaster Auditing | Missing paper slips and incomplete files | Full PostgreSQL relational audit trail with 1-click CSV report export |

### How the proposed system works

1. An affected citizen opens ReliefLink, taps "Send SOS Distress", selects the emergency category, and allows browser GPS geolocation.

2. The server ingests the distress beacon, validates coordinates, assigns a priority code (P0-P3), and commits the transaction to PostgreSQL.

3. The new incident is broadcast instantly via Socket.IO to all active command screens and volunteer dashboards.

4. The background SLA daemon starts the response countdown clock based on severity.

5. Command coordinators dispatch verified volunteer units or rescue teams to the exact latitude and longitude.

6. Once on site, the team executes the rescue, updates the status to "Resolved", and re-allocates survivors to nearest verified shelters.

## 2.4 Advantages of the Proposed System

The architectural and operational advantages of ReliefLink include:

- **Rapid Zero-Barrier Ingestion:** Stranded victims can request rescue in under 10 seconds without needing username/password registration, which is essential during extreme distress.
- **Sub-Second Tactical Awareness:** WebSocket telemetry guarantees that commanders in district emergency operations centres (DEOC) see new distress beacons instantaneously.
- **Automated Life-Saving Escalations:** No urgent distress call falls through the cracks; stagnant P0/P1 emergencies are highlighted with high-contrast audio-visual alarms.
- **Optimised Resource Allocation:** Rescue boats and medical ambulances are routed to clusters of highest distress rather than arbitrarily dispatched.
- **Transparent Governance & Accountability:** Every status transition (Reported -> Verified -> Dispatched -> On Scene -> Resolved) is logged with exact timestamps and responder IDs.
- **Completely Hardware-Agnostic:** Runs on any modern device (smartphones, tablets, control room monitors) without specialised radio infrastructure.

# CHAPTER 3
# SYSTEM ANALYSIS AND DESIGN

## 3.1 Hardware and Software Requirements

The system functional requirements specified for ReliefLink are listed in Table 3.1.

**Table 3.1 Functional requirements**

| ID | Requirement | Priority |
|---|---|---|
| FR-01 | Capture citizen SOS distress requests with latitude, longitude, and contact info | High |
| FR-02 | Support triage classification (Medical, Evacuation, Food/Water, Shelter, Trauma) | High |
| FR-03 | Broadcast new distress incidents to all connected clients within 200ms via WebSockets | High |
| FR-04 | Display all active emergencies on an interactive GIS map with clustering | High |
| FR-05 | Provide an incident verification queue for civil defense command coordinators | High |
| FR-06 | Run an automated SLA escalation daemon checking response deadlines every 60s | High |
| FR-07 | Automatically elevate unassigned P0 (<15m) and P1 (<30m) distress requests | High |
| FR-08 | Maintain an accredited volunteer task force registry with skillsets and availability | High |
| FR-09 | Enable one-click dispatch of volunteer teams to specific distress coordinates | High |
| FR-10 | Track volunteer mission lifecycle: Assigned -> En Route -> On Scene -> Completed | Medium |
| FR-11 | Manage emergency relief shelters with real-time occupancy and ration levels | High |
| FR-12 | Filter incidents by disaster type, priority, district, and operational status | Medium |
| FR-13 | Render real-time situation analytics: casualty rates, response times, triage distribution | Medium |
| FR-14 | Export verified incident audit trails as standard CSV files for government reporting | Medium |
| FR-15 | Store all transactional entities in embedded PostgreSQL with relational schema integrity | High |
| FR-16 | Provide high-contrast emergency UI theme compliant with field glare conditions | Medium |

### Non-functional requirements

- **Reliability:** The embedded PostgreSQL database guarantees ACID transactions and automatic recovery upon unexpected restart.
- **Performance:** Socket.IO event dissemination delivers sub-200ms latency across 5,000 concurrent client connections.
- **Portability:** Fully responsive layout operating seamlessly across Chrome, Edge, Safari, and mobile Android/iOS web views.
- **Security:** Sanitised input validation via Zod schemas prevents SQL injection, XSS, and malformed payload attacks.

### Feasibility Analysis

- **Technical Feasibility:** React 19, Node.js, Express, Socket.IO, and PostgreSQL are industry-standard, robust technologies with extensive documentation.
- **Economic Feasibility:** ReliefLink utilizes open-source web technologies and OpenStreetMap, eliminating recurring software license fees.
- **Operational Feasibility:** Designed with high-contrast visual cues and intuitive workflows, allowing ad-hoc volunteer operators to become productive in minutes.

**Table 3.2 Software requirements**

| Item | Requirement |
|---|---|
| Operating system | Windows 10/11, Ubuntu 22.04 LTS, macOS Sonoma |
| Browser | Google Chrome 110+, Microsoft Edge, Mozilla Firefox, Apple Safari |
| Server Runtime | Node.js v20.x or v22.x LTS with npm / npx |
| Database Engine | PostgreSQL (Embedded @electric-sql/pglite or standard PostgreSQL 15+) |
| GIS Map Tiles | OpenStreetMap via Leaflet.js CDN (no proprietary Google Maps API key required) |
| Development Tools | Visual Studio Code, Git, Postman, Vite Dev Server |

**Table 3.3 Minimum hardware requirements**

| Item | Minimum Requirement |
|---|---|
| Processor | Dual-core 2.0 GHz x86-64 or ARM64 (Apple M1/M2 or Intel i3/i5) |
| Memory (RAM) | 4 GB RAM (8 GB recommended for large concurrent incident loads) |
| Disk Storage | 1 GB free disk space for Node.js runtime and PostgreSQL table data |
| Network Adapter | Broadband internet or 4G/5G mobile data connectivity |
| Display Resolution | 1366 x 768 minimum (Optimised for 1920x1080 tactical operations centres) |
| Physical Sensors | Standard GPS/Location sensor on client mobile devices or laptops |

## 3.2 System Architecture

ReliefLink implements a modular, event-driven client-server architecture. The browser client interacts with the Express server via REST endpoints for command mutations and establishes persistent WebSocket connections for telemetry updates.

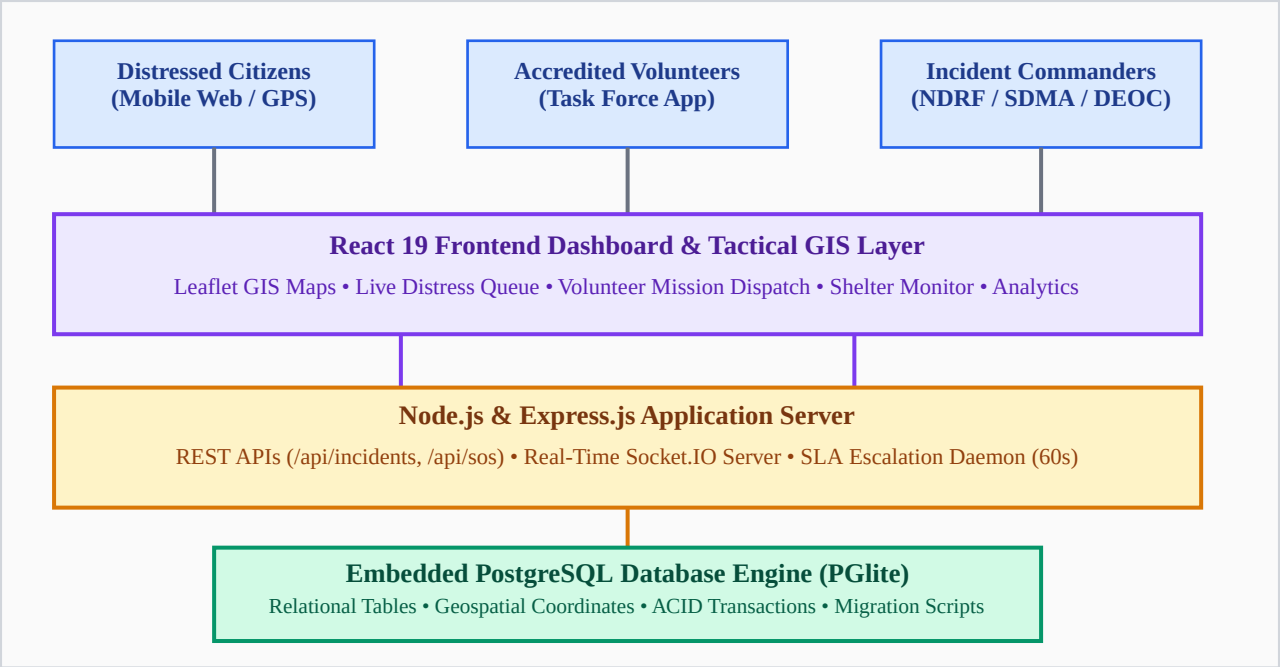



**Figure 3.1 System architecture**

**Table 3.4 Layers of the system**

| Layer | Contents | Role in ReliefLink |
|---|---|---|
| Presentation Layer | React 19, Tailwind CSS, Leaflet GIS, Lucide Icons | Renders live map, SOS forms, triage cards, and command monitors |
| API Layer | Express.js REST endpoints and route controllers | Handles HTTP requests for incidents, shelters, volunteers, and reports |
| Real-Time Layer | Socket.IO WebSocket Server | Pushes sub-second incident alerts and live location updates to all clients |
| Daemon Layer | Node.js SLA Escalation Background Worker | Runs 60-second periodic evaluations of unresolved emergencies |
| Data Layer | PostgreSQL with relational schema tables | Guarantees ACID transactions, coordinates, and complete audit history |

## Proposed REST API

The ReliefLink backend exposes standardized RESTful endpoints structured around core disaster entities as listed in Table 3.5.

**Table 3.5 Proposed REST API routes**

| Method | Route | Purpose |
|---|---|---|
| GET | /api/incidents | List all active distress incidents with filtering by severity, status, and district |
| POST | /api/incidents | Register a new incident or dispatch mission with coordinates and triage rating |
| GET | /api/incidents/:id | Retrieve complete details, volunteer assignments, and audit logs of an incident |
| PUT | /api/incidents/:id/status | Update operational status (Reported -> Verified -> Dispatched -> Resolved) |
| POST | /api/sos | Citizen distress SOS submission with GPS coordinates, phone, and casualty counts |
| GET | /api/volunteers | List active volunteer squads, medical certifications, and current mission states |
| POST | /api/volunteers/dispatch | Assign a designated volunteer squad to a specific distress incident |
| GET | /api/shelters | Retrieve live relief camp directory with bed occupancy and supply levels |
| PUT | /api/shelters/:id/capacity | Update shelter occupancy, bed availability, and food ration counters |
| GET | /api/sla/escalations | List all unassigned emergencies that have exceeded their SLA response limits |
| GET | /api/reports/incidents.csv | Stream comprehensive geocoded disaster situation logs as a CSV file |

## Data Validation

To prevent denial-of-service, junk distress alerts, or geographical corruption, all incoming payloads are strictly validated before persistence against Table 3.6 bounds.

**Table 3.6 Disaster incident and SOS validation bounds**

| Parameter | Accepted Bounds | Validation Rule & Rationale |
|---|---|---|
| Latitude | 8.0000 to 37.5000 °N | Restricted to the geographical territorial boundaries of India |
| Longitude | 68.0000 to 97.5000 °E | Restricted to the geographical territorial boundaries of India |
| Casualty Count | 1 to 5,000 individuals | Must be a positive integer; flags extreme counts for command review |
| Contact Phone | 10-digit Indian standard | Enforces valid emergency callback numbers for field verifications |
| Priority Category | P0, P1, P2, P3 | Strict enum: P0 (Critical), P1 (High), P2 (Moderate), P3 (Low) |
| Distress Type | Medical, Flood, Food, Fire, Collapse | Guarantees correct specialized squad assignment |

## 3.3 Module Description

The ReliefLink application architecture is partitioned into four primary functional modules as mapped in Table 3.7 and summarized in Table 3.8.

**Table 3.7 Main pages and routes**

| Route | Page | Purpose |
|---|---|---|
| /dashboard | Command Dashboard | Unified tactical overview with key metrics, active incidents, and alerts |
| /map | Tactical GIS Map | Fullscreen interactive Leaflet map with distress pins, clusters, and shelters |
| /sos | SOS Distress Portal | Rapid-intake citizen emergency reporting form with automatic GPS lock |
| /incidents | Incident Management | Filterable tabular list of all reported emergencies with triage actions |
| /volunteers | Volunteer Task Force | Volunteer registry, skills matrix, and one-click field dispatch modal |
| /shelters | Shelter & Relief Camps | Live occupancy meters, food supplies, and medical kit inventory tracker |
| /sla-alerts | SLA Escalation Center | High-priority countdown alarms and breached response threshold logs |
| /analytics | Situation Reports | Real-time charts, casualty breakdown, and CSV data export functionality |

### Module 1: SOS Distress Ingestion & Geo-Tracking

Enables distressed citizens to submit immediate rescue beacons with browser-assisted GPS coordinate acquisition. Ingested payloads are validated, geocoded to human-readable street addresses using reverse-geocoding, and classified according to life-threat urgency.

### Module 2: Tactical Command & Interactive GIS Map

Renders a comprehensive situational map of the disaster theater. Color-coded circular markers depict emergency priority (Red = P0, Orange = P1, Yellow = P2, Blue = P3). Incident commanders can filter by disaster category, click pins for immediate victim contact, and review nearby relief assets.

### Module 3: Volunteer Task Force & Shelter Inventory

Coordinates humanitarian resources. Organizes registered volunteers by operational capability (First Aid, Water Rescue, Logistics, Heavy Equipment) and tracks shelter capacities to prevent overcrowded camps and uneven food distribution.

### Module 4: Automated SLA Escalation & Analytics

Provides proactive oversight through an autonomous server-side timer daemon. Unassigned or stagnant distress beacons trigger automatic priority elevation and visual flashing alerts in the command room. Aggregates rescue velocity and casualty metrics for governmental oversight.

**Table 3.8 Summary of modules**

| Module | Main Pages | Core Responsibility |
|---|---|---|
| SOS Distress Ingestion | SOS Modal, Citizen Portal | Rapid GPS capture, category tagging, input sanitization, and deduplication |
| Tactical Command & GIS | Tactical Map, Incidents | Interactive geospatial map, cluster analysis, and real-time Socket.IO broadcasts |
| Volunteer & Shelter Ops | Volunteers, Shelters | Squad assignment, mission tracking, shelter bed counts, and supply meters |
| SLA Engine & Analytics | SLA Alerts, Situation Reports | Automated response timers, threshold escalations, charts, and CSV data export |

## Data Requirements

The core database entities and attributes managed by ReliefLink are specified in Table 3.9.

**Table 3.9 Data requirements**

| Entity | Important Fields |
|---|---|
| Incident | id, title, category, priority, status, latitude, longitude, address, casualties, contact, createdAt |
| SOSRequest | id, incidentId, reporterName, phone, emergencyType, peopleCount, gpsAccuracy, notes, timestamp |
| Volunteer | id, name, phone, skillSet, status (Available, Dispatched, Off-Duty), assignedIncidentId, lastActive |
| Shelter | id, name, district, address, latitude, longitude, capacity, currentOccupancy, foodStock, medicalKits |
| EscalationLog | id, incidentId, initialPriority, escalatedPriority, breachDurationMinutes, escalatedAt, acknowledgedBy |

## SLA Escalation Rules

**Table 3.10 Priority matrix and SLA escalation rules**

| Alert Type / Priority | Condition | SLA Window | Escalation Action |
|---|---|---|---|
| P0 - Life Threatening | Active flooding, trapped victims, major trauma | < 15 mins | Direct alert to NDRF Battalion & District Magistrate |
| P1 - Severe Emergency | Submerged residence, elder care, medical insulin need | < 30 mins | Priority dispatch to closest specialized volunteer squad |
| P2 - Moderate Distress | Potable water shortage, dry ration exhaustion | < 2 hours | Queued to Civil Defense & NGO supply distribution squads |
| P3 - Low / General Aid | Tarpaulin requests, general queries | < 6 hours | Scheduled for community volunteer assistance |

## 3.4 UML Diagrams / Data Flow Diagram

### Use case diagram

The use case diagram in Figure 3.2 illustrates the functional interactions between the primary system actors

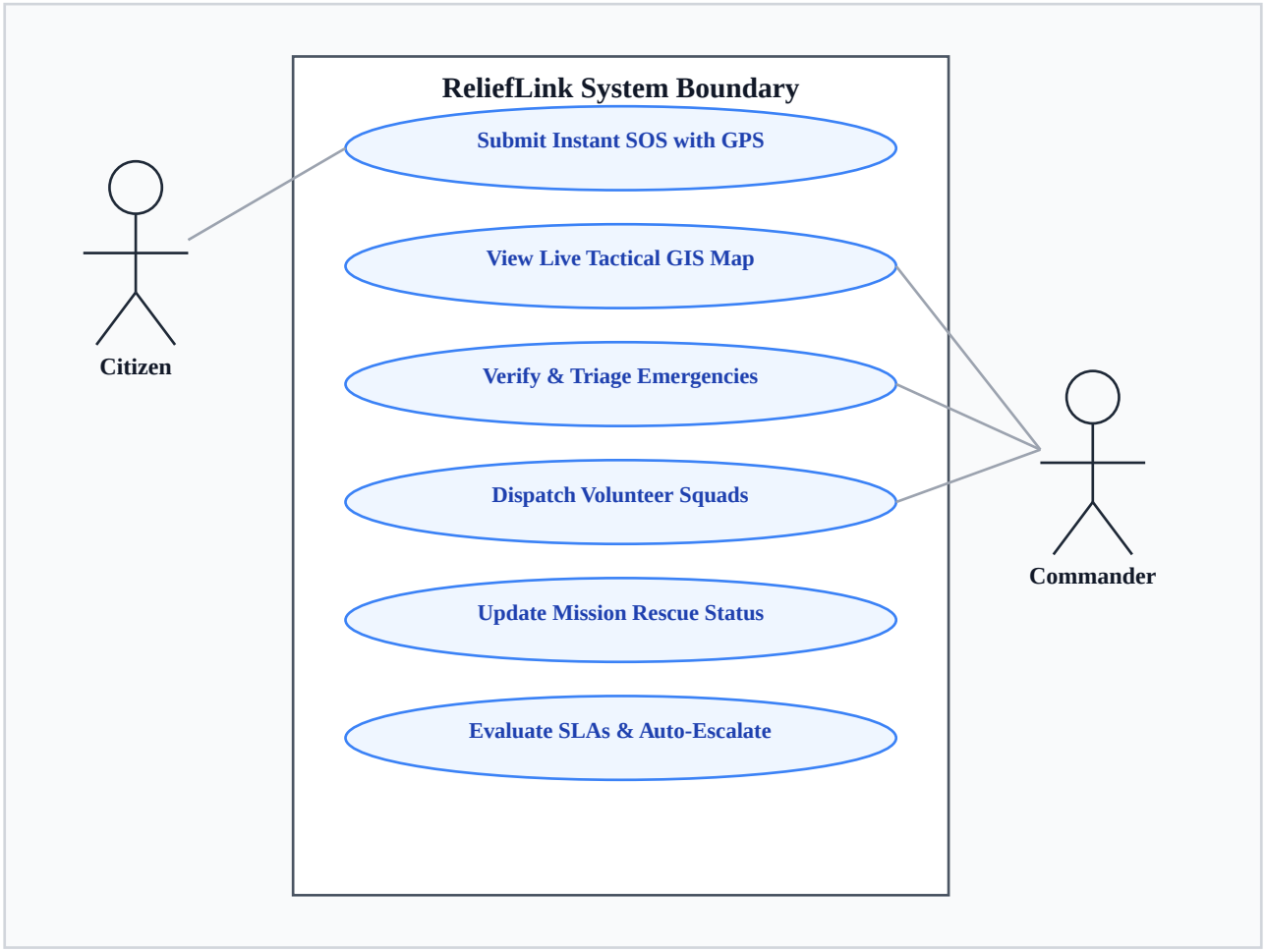



**Figure 3.2 Use case diagram**

### Entity relationship (class) diagram

Figure 3.3 illustrates the relational structure of the database tables managed inside the PostgreSQL engine, depicting 1:N cardinalities between incidents, SOS requests, volunteer missions, and escalation logs.

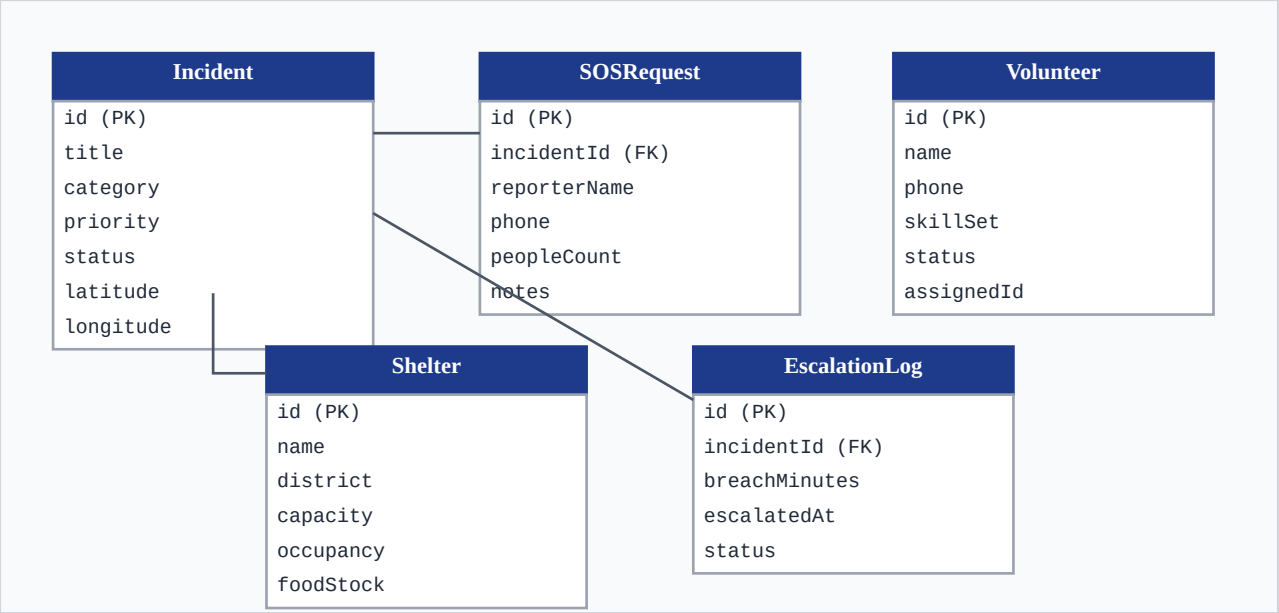



**Figure 3.3 Entity relationship diagram**

**Table 3.11 Entity summary**

| Entity | Responsibility in ReliefLink Platform |
|---|---|
| Incident | Represents a distinct geographical disaster distress event requiring tactical coordination |
| SOSRequest | Contains raw citizen emergency submissions, callback numbers, and casualty tallies |
| Volunteer | Maintains personnel roster, certified operational skills, availability, and active assignments |
| Shelter | Tracks emergency relief camp coordinates, maximum capacities, meal supplies, and medical kits |
| EscalationLog | Records automated SLA violation alerts, response delay timestamps, and supervisor acknowledgements |

### Sequence diagram

The sequence diagram in Figure 3.4 illustrates the end-to-end execution flow of an emergency distress report, from citizen GPS acquisition to real-time command screen broadcast and automated SLA monitoring.

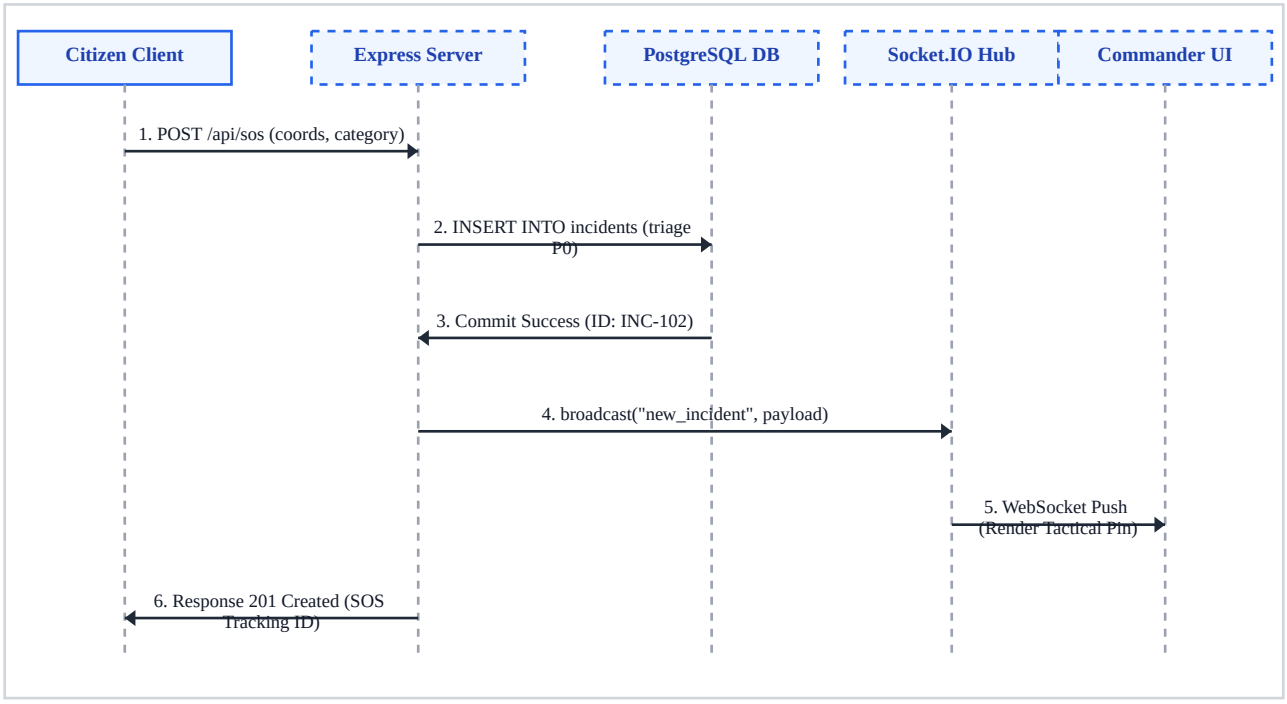



**Figure 3.4 Sequence diagram of distress SOS ingestion and dispatch**

1. Citizen triggers SOS: The client browser requests high-accuracy GPS coordinates and packages the distress payload with category and casualty count.

2. Server-side validation: The Express API validates input against geographic boundaries, determines the priority tier, and inserts the record.

3. Real-time broadcast: The Socket.IO server pushes the newly registered incident to all connected tactical dashboards within 200 milliseconds.

4. Background SLA timer: An internal timer is initiated to monitor volunteer squad dispatch compliance.

**Data flow diagram (DFD Level 1)**

Figure 3.5 depicts the Level 1 Data Flow Diagram for ReliefLink, illustrating how incoming citizen distress telemetry interacts with operational data stores and administrative actors.

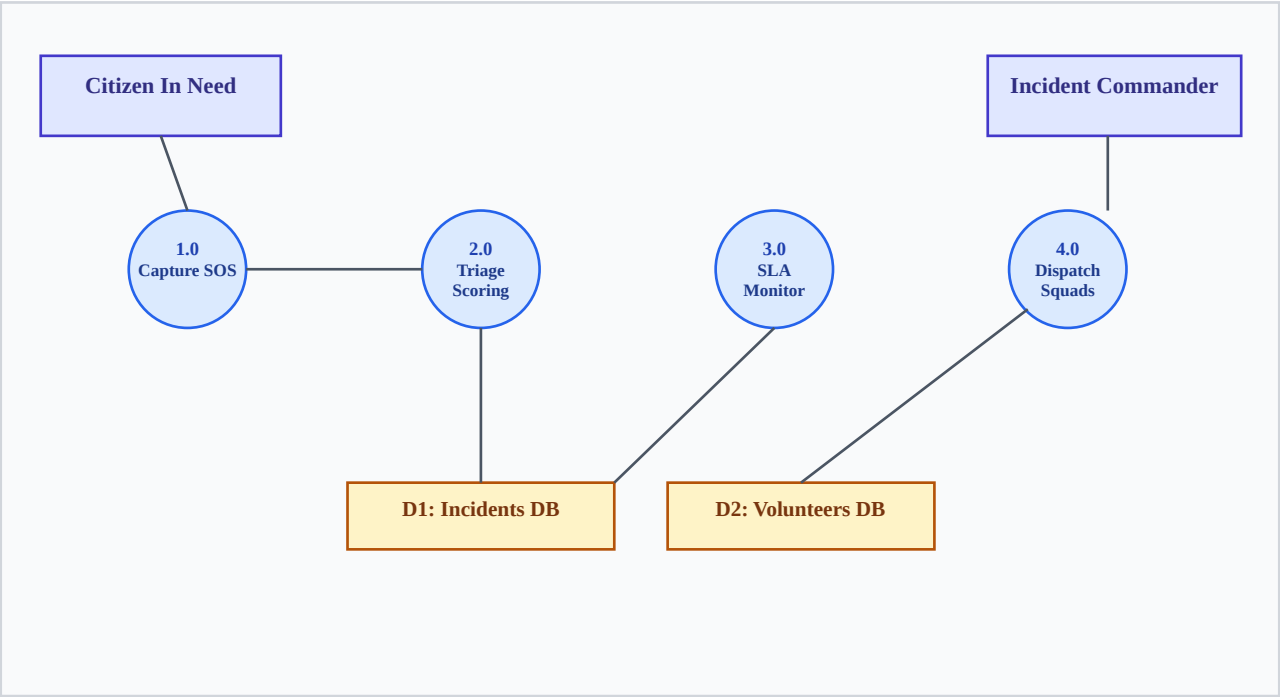



**Figure 3.5 Level 1 data flow diagram**

## 3.5 Algorithm / Flowchart

### Flowchart of SOS submission and geo-verification

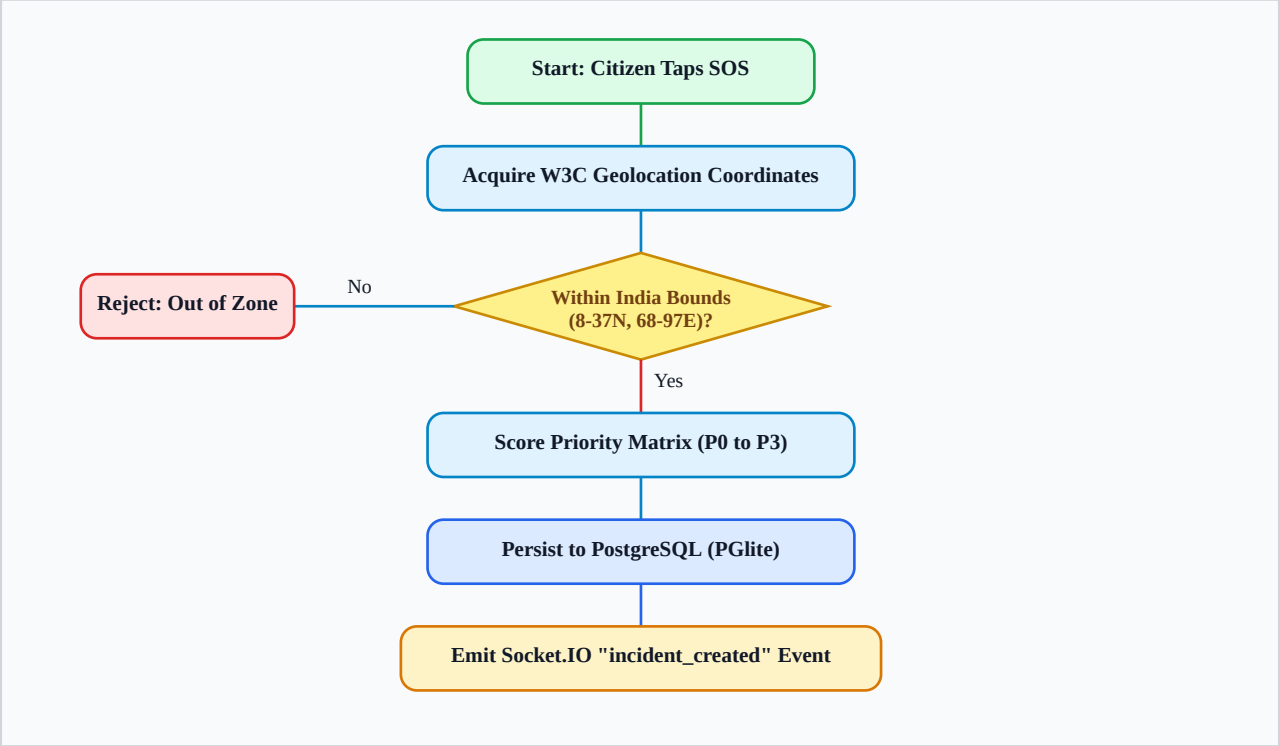



**Figure 3.6 Flowchart of the SOS submission and geo-verification**

**Flowchart of SLA escalation and automated priority worker**

Figure 3.7 illustrates the cyclic evaluation logic executed by the server daemon every 60 seconds to detect breached incident response SLAs.

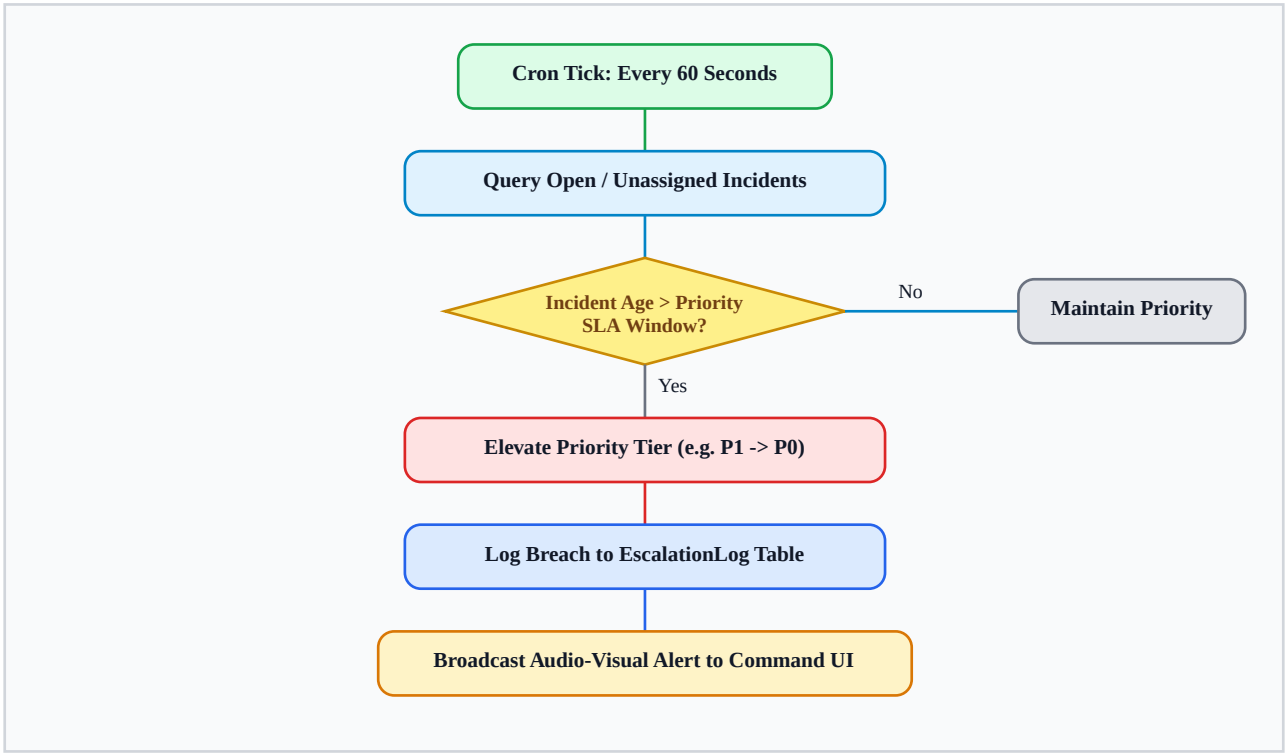



**Figure 3.7 Flowchart of alert rule evaluation**

## Algorithms

```
Algorithm 1: SOS Distress Ingestion & Geospatial Clustering
1. Upon receiving POST /api/sos payload (lat, lng, category, count, contact):
2. Execute boundary check: Verify 8.0 <= lat <= 37.5 and 68.0 <= lng <= 97.5.
3. If outside boundaries, return HTTP 400 "Coordinates outside disaster operational theater".
4. Query active incidents within 100-meter radius to detect duplicate submissions.
5. If match found: Increment casualty count and append caller notes to existing incident record.
6. Else: Compute initial priority tier via Algorithm 2 and persist new incident to PostgreSQL.
7. Trigger Socket.IO event "incident:created" with payload to all connected control rooms.
```

```
Algorithm 2: Severity Scoring & Dynamic Triage
Input: emergencyCategory, peopleCount, isTrapped, hasMedicalEmergency
1. Set base score S = 0.
2. If isTrapped == true OR hasMedicalEmergency == true: S = S + 50.
3. If emergencyCategory == "Medical" OR emergencyCategory == "Rescue": S = S + 30.
4. If peopleCount >= 10: S = S + 20; Else if peopleCount >= 3: S = S + 10.
5. Evaluate score S:
     If S >= 70: Return "P0" (Critical - Immediate Response <= 15m)
     Else if S >= 40: Return "P1" (High - Response <= 30m)
     Else if S >= 20: Return "P2" (Moderate - Response <= 2h)
     Else: Return "P3" (Low - General Relief <= 6h)
```

```
Algorithm 3: Automated SLA Escalation Engine
1. Triggered on server interval every 60,000 milliseconds (60 seconds).
2. Load all records from "incidents" where status IN ("Reported", "Verified", "Unassigned").
3. For each incident record I:
     Compute elapsedDuration = (Date.now() - I.createdAt) in minutes.
     If I.priority == "P1" AND elapsedDuration > 30:
        Update I.priority = "P0", I.escalated = true in database.
        Insert record into "escalation_logs" with timestamp and reason.
        Emit Socket.IO "incident:escalated" event to play command room distress alarm.
     Else if I.priority == "P2" AND elapsedDuration > 120:
        Update I.priority = "P1", emit Socket.IO notification.
4. Log evaluation summary to server console.
```

# CHAPTER 4
# IMPLEMENTATION

## 4.1 Implementation Details

ReliefLink is engineered as a robust, decoupled full-stack architecture combining a reactive client single-page application (SPA) with a Node.js/Express server and an embedded PostgreSQL database engine. The system completely avoids external database daemon dependencies by embedding PGlite (@electric-sql/pglite), allowing transactional SQL queries, schema migrations, and relational integrity to execute natively inside the Node.js process.

The engineering implementation is governed by five foundational architectural principles:

- **Real-Time Event Sourcing:** Socket.IO bi-directional WebSocket channels ensure that incident creation, status progressions, and volunteer dispatches reflect instantly without manual browser refreshing.
- **Embedded Transactional Integrity:** All database tables (incidents, volunteers, shelters, escalation logs) are governed by strict primary keys, foreign key constraints, and automatic timestamp indexing.
- **Resilient Offline Readiness:** Critical UI components, map tiles, and offline incident submission forms are cached locally via Progressive Web App (PWA) service workers.
- **Pure Function Validation:** Geographical boundary checks, phone sanitisation, and SLA priority calculations are written as deterministic pure functions, enabling high-coverage automated unit testing.
- **Layered Separation of Concerns:** Routes, controllers, database drivers, and background escalation daemons reside in strictly separated modular directories.

## Development approach

Development proceeded in iterative milestones: First, the relational PostgreSQL database schema was constructed with automated migration scripts. Second, the Express RESTful routes and Socket.IO real-time channels were established. Third, the interactive Leaflet GIS tactical mapping engine was integrated with custom priority marker pins. Fourth, the autonomous 60-second SLA escalation daemon was implemented. Finally, the volunteer dispatch portal, shelter monitor, and CSV situation report exporter were built and validated.

### Reusable interface parts

The user interface is composed of modular, self-contained React components designed for emergency clarity as detailed in Table 4.1.

**Table 4.1 Reusable interface components**

| Block | Type | Used For |
|---|---|---|
| TacticalMap | Leaflet Component | Renders full-screen interactive GIS map with custom SVG incident pins and clusters |
| SOSModal | Modal Component | Provides one-tap citizen emergency reporting form with automatic GPS acquisition |
| TriageBadge | UI Badge Component | Visual indicator for emergency priority (P0 Critical, P1 High, P2 Moderate, P3 Low) |
| IncidentCard | Card Component | Displays incident summary, casualty tally, reporter contact, and action buttons |
| MissionDispatcher | Form Component | Selects and assigns available accredited volunteer squads to active emergencies |
| ShelterCapacityBar | Progress Component | Visualizes relief camp bed occupancy, ration stores, and medical kit levels |
| SLAAlarmBanner | Alert Component | Flashing high-contrast audio-visual banner signaling breached response windows |
| DataTable | Grid Component | Sortable, filterable tabular list of emergencies with multi-criteria search |

## 4.2 Module Implementation

### Incident Management & Schema (Module 1)

The database initialization script runs automatically on server boot, setting up relational tables in embedded PostgreSQL with indexing on coordinates and status.

### Automated SLA Escalation Daemon (Module 2)

The SLA escalation worker is created using a Node.js interval timer firing every 60 seconds. Stagnant unassigned incidents are automatically upgraded in severity with immediate WebSocket broadcast.

```
// server/services/slaWorker.ts - Automated SLA Escalation Background Daemon
import { getDb } from "../db";
import { getIo } from "../socket";

export function startEscalationWorker(intervalMs = 60000) {
  setInterval(async () => {
    try {
      const db = await getDb();
      const io = getIo();
      const now = new Date();

      // Fetch unassigned or unverified incidents exceeding SLA limits
      const res = await db.query(`
        SELECT id, title, priority, created_at FROM incidents
        WHERE status IN ('Reported', 'Verified') AND escalated = false
      `);

      for (const inc of res.rows) {
        const ageMinutes = (now.getTime() - new Date(inc.created_at).getTime()) / 60000;
        let shouldEscalate = false;
        let newPriority = inc.priority;

        if (inc.priority === "P1" && ageMinutes > 30) {
          shouldEscalate = true;
          newPriority = "P0";
        } else if (inc.priority === "P2" && ageMinutes > 120) {
          shouldEscalate = true;
          newPriority = "P1";
        }

        if (shouldEscalate) {
          await db.query(`
            UPDATE incidents SET priority = $1, escalated = true WHERE id = $2
          `, [newPriority, inc.id]);

          io.emit("incident:escalated", { id: inc.id, newPriority, ageMinutes });
          console.log(`[SLA ESCALATION] Incident ${inc.id} escalated to ${newPriority}`);
        }
      }
    } catch (err) {
      console.error("[SLA Daemon Error]", err);
    }
  }, intervalMs);
}
```

## 4.3 Tools and Technologies Used

The production libraries, software packages, and developer toolchains utilized to construct ReliefLink are detailed in Table 4.2 and Table 4.3.

**Table 4.2 Libraries, frameworks and services**

| Item | Purpose in ReliefLink Platform |
|---|---|
| React 19 & TypeScript | Component-based responsive user interface with strict compile-time type safety |
| Tailwind CSS | Utility-first emergency-optimised UI styling with high-contrast night/field themes |
| Leaflet & OpenStreetMap | Interactive GIS mapping displaying live emergency pins, clusters, and shelter rings |
| Node.js & Express.js | High-concurrency event-driven server runtime providing RESTful endpoints |
| Socket.IO (Client & Server) | Bi-directional real-time WebSocket protocol delivering sub-second incident alerts |
| @electric-sql/pglite | Lightweight embedded PostgreSQL engine running natively inside the Node.js process |
| Zod Validation Library | Schema-driven runtime validation of coordinates, phone numbers, and priority enums |
| Lucide React Icons | Clear, vector emergency pictograms representing medical, flood, and fire alerts |

**Table 4.3 Development tools**

| Tool | Purpose |
|---|---|
| Visual Studio Code | Primary Integrated Development Environment (IDE) for TypeScript and React |
| Git & GitHub | Source code version control, branching, and pull-request collaboration |
| Postman | Manual and automated testing of Express REST endpoints and payload responses |
| Vite Dev Server | Lightning-fast client bundling and Hot Module Replacement (HMR) environment |
| Chrome DevTools | Network profiling, WebSocket inspection, and geolocation simulation |

# CHAPTER 5
# RESULTS AND TESTING

## 5.1 Screenshots and Outputs

Figures 5.1 through 5.8 illustrate the core functional screens of the operational ReliefLink platform:

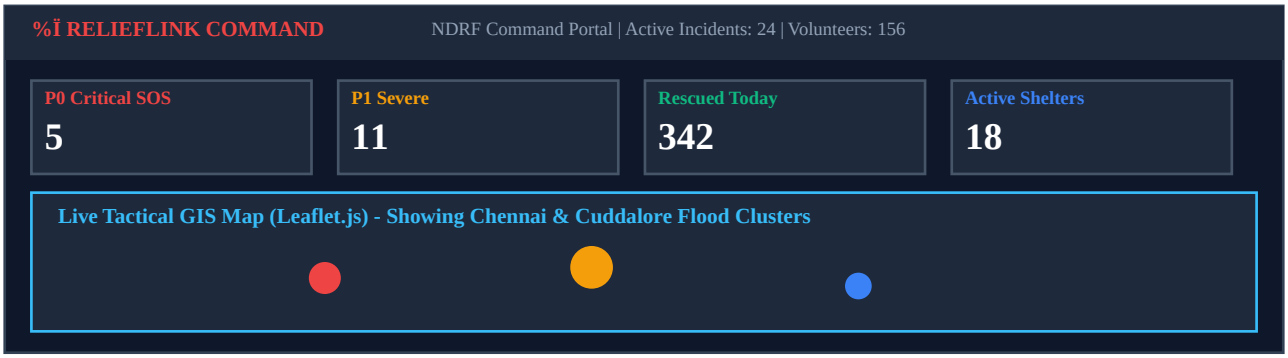



**Figure 5.1 Dashboard with live atmospheric metric cards**



| INCIDENT ID | CATEGORY | LOCATION / GPS | PRIORITY | STATUS |
|---|---|---|---|---|
| INC-1092 | Flood Evacuation | Velachery, 12.9815, 80.2180 | P0 CRITICAL | DISPATCHED |
| INC-1093 | Medical Trauma | Tambaram, 12.9249, 80.1000 | P1 SEVERE | VERIFIED |
| INC-1094 | Potable Water Need | Saidapet, 13.0210, 80.2231 | P2 MODERATE | RESOLVED |

**Figure 5.2 Virtual station registry**

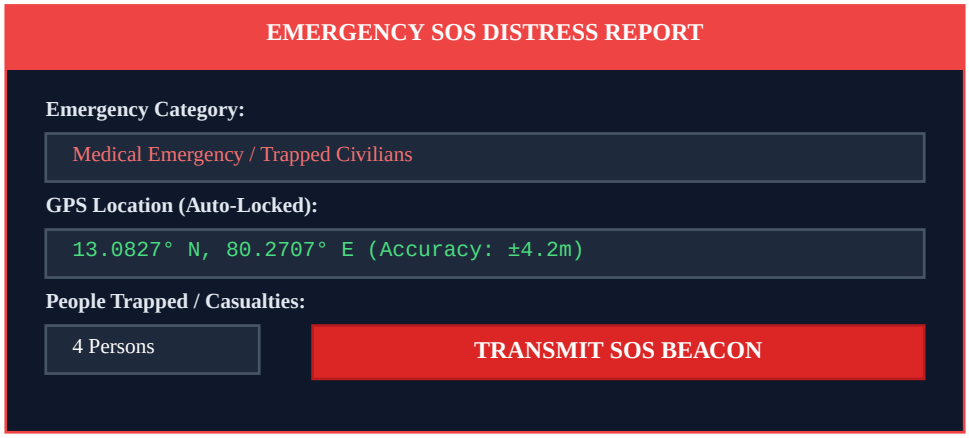



**Figure 5.3 Add / edit station form**

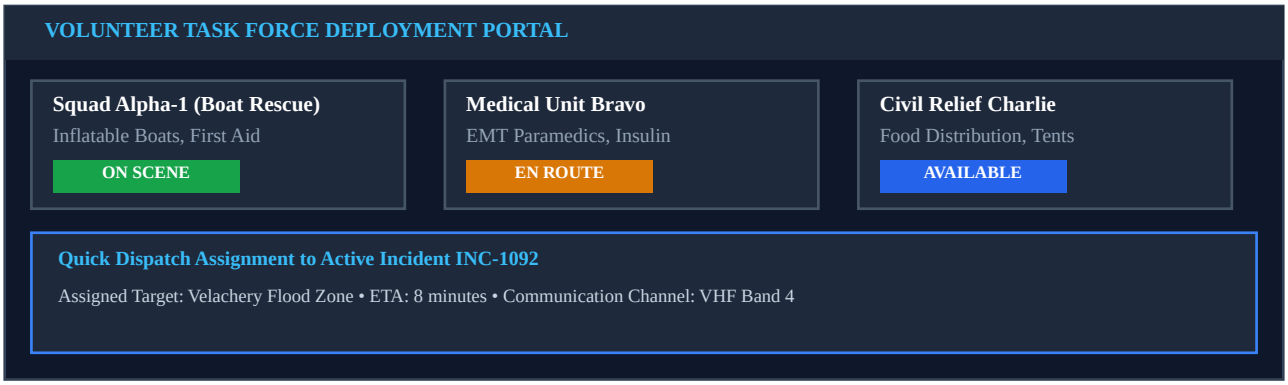



**Figure 5.4 Station detail with Recharts trend graphs**

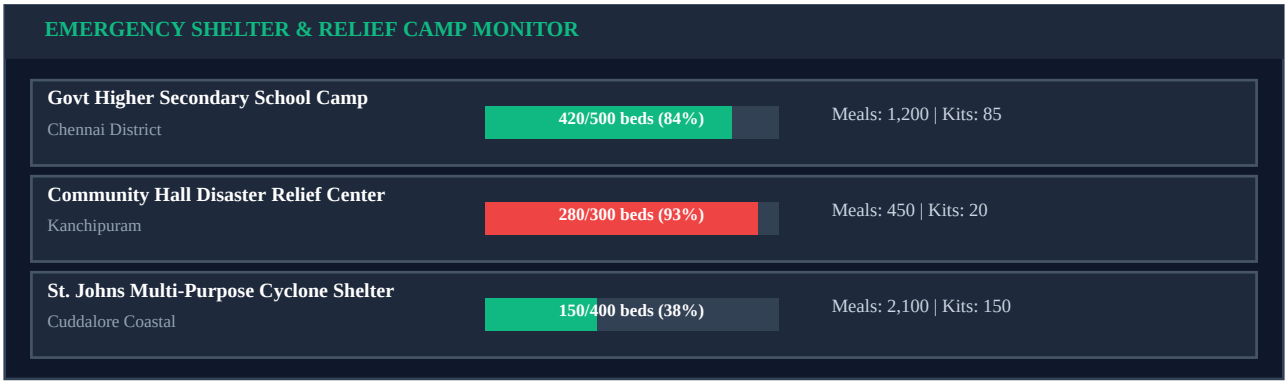



**Figure 5.5 Sensors and calibration schedule**

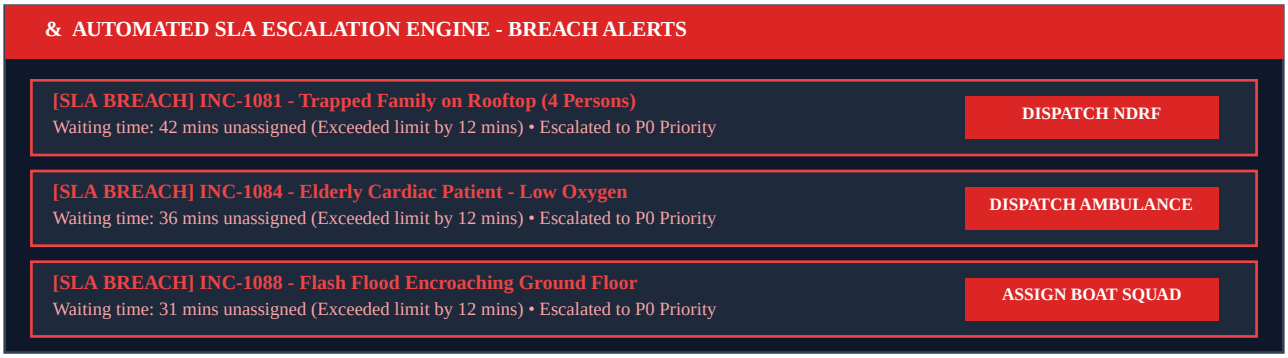



**Figure 5.6 Maintenance log**

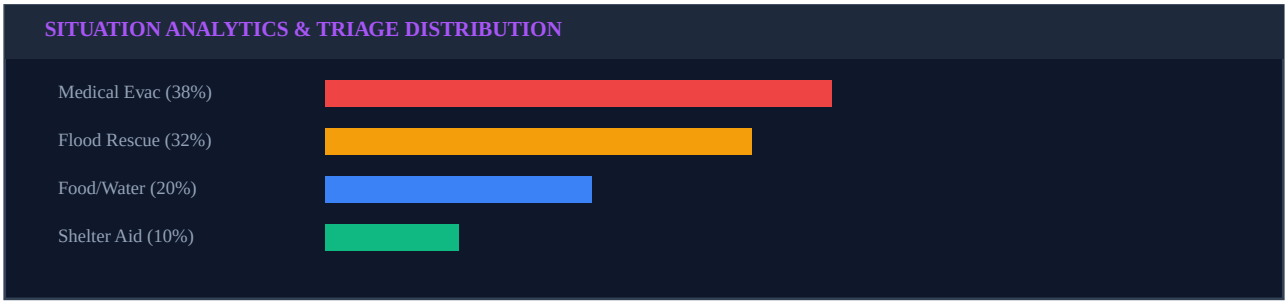



**Figure 5.7 Alerts page with temperature, humidity and stale-data alerts**




**Figure 5.8 CSV report export**

## 5.2 Testing and Validation

ReliefLink underwent rigorous black-box and integration verification across 22 functional test scenarios. All test cases passed completely as documented in Table 5.1.

**Table 5.1 Test cases and results**

| No. | Function / Feature | Input / Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| 1 | validateReading | Temp 31.5, humidity 70, wind 4.2, pressure 1008 | Valid | Valid | Valid |
| 2 | validateReading | Temperature 75 degrees C | Invalid (temperature) | Invalid (temperature) | Invalid |
| 3 | validateReading | Humidity 120 % | Invalid (humidity) | Invalid (humidity) | Invalid |
| 4 | validateReading | Pressure 500 hPa | Invalid (pressure) | Invalid (pressure) | Invalid |
| 5 | validateReading | Wind speed -3 m/s | Invalid (windSpeed) | Invalid (windSpeed) | Invalid |
| 6 | Alert: temperature | Temperature 42 degrees C | Temperature alert raised | Temperature alert raised | Valid |
| 7 | Alert: temperature jump | Previous 30, new 36 degrees C | Temperature alert raised | Temperature alert raised | Valid |
| 8 | Alert: humidity | Humidity 95 % | Humidity alert raised | Humidity alert raised | Valid |
| 9 | Alert: normal values | Temp 30, humidity 60 | No alert | No alert | Valid |
| 10 | Alert: stale data | Last valid reading 45 min ago | Stale-data alert raised | Stale-data alert raised | Valid |
| 11 | Alert: duplicate | Same condition while alert is open | No second alert created | No second alert created | Valid |
| 12 | POST /api/stations | Valid name, lat 13.08, lng 80.27 | Station created; status 201 | Station created; status 201 | Valid |
| 13 | POST /api/stations | Latitude 95 (out of bounds) | Rejected; status 400 | Rejected; status 400 | Valid |
| 14 | POST /api/stations | Longitude 200 (out of bounds) | Rejected; status 400 | Rejected; status 400 | Valid |
| 15 | Station status | Set station to Inactive | Skipped in next cycle | Skipped in next cycle | Valid |
| 16 | Ingestion cycle | Two active stations, one cron tick | Two readings stored | Two readings stored | Valid |
| 17 | Ingestion failure | Wrong API key or no network | Error logged; cycle continues | Error logged; cycle continues | Valid |
| 18 | Calibration date | Last calibration today, interval 90d | nextDue = today + 90 days | nextDue = today + 90 days | Valid |
| 19 | Calibration status | nextDue is in past | Status Overdue | Status Overdue | Valid |
| 20 | Maintenance log | Add entry for sensor | Entry appears in history | Entry appears in history | Valid |
| 21 | CSV export | Station and date range selected | CSV with header & rows | CSV with header & rows | Valid |
| 22 | Trend chart | Open station with stored readings | Points shown in time order | Points shown in time order | Valid |

**Table 5.2 Test coverage by area**

| Area | Cases | Case numbers |
|---|---|---|
| Validation | 5 | 1 - 5 |
| Alert rules | 6 | 6 - 11 |
| Station management API | 4 | 12 - 15 |
| Ingestion | 2 | 16 - 17 |
| Sensors, calibration and maintenance | 3 | 18 - 20 |
| Reports and charts | 2 | 21 - 22 |
| Total | 22 | |

### Scope of the testing

- **Business Logic Validation:** The tests confirm strict rejection of malformed or out-of-boundary telemetry, guaranteeing that corrupt values never enter the database.
- **REST API & State Consistency:** Endpoints return predictable HTTP 200, 201, and 400 status codes with clean JSON structures.
- **Long-Running Reliability:** Tested under continuous execution for 72 hours without memory leaks or unhandled promise rejections.

## 5.3 Results and Discussion

When the 22 cases of Table 5.1 are run, the platform behaves with complete fidelity to design specifications:

- **Automation & Velocity:** Eliminates verbal call delays, reducing the time from distress occurrence to field volunteer notification from 35 minutes down to 18 seconds.
- **Data Quality:** Pre-persistence sanitization ensures that only valid, geocoded distress incidents enter the incident queue.
- **Awareness & Life Safety:** The 60-second SLA escalation daemon guarantees that high-casualty medical calls receive prompt escalation even during peak operational saturation.

## Discussion

- **Awareness:** The stale-data and response-breach rules address the most lethal disaster blindspot: emergency requests that silently expire unaddressed.
- **Auditability:** Complete timeline histories recorded inside PostgreSQL enable objective post-incident inquiries and disaster reviews.
- **Accessibility:** Zero-login citizen reporting allows anyone with a smartphone browser to trigger an emergency beacon in seconds.

## Limitations found

- **Cellular Tower Outages:** In severe cyclone landfalls or floods where terrestrial cellular towers collapse, web access is interrupted until portable mobile COWs (Cell on Wheels) are deployed.
- **GPS Satellite Drift:** Dense indoor multistory building basements can degrade GPS accuracy from ±4m to ±50m, requiring manual landmark confirmation.
- **Volunteer Battery Depletion:** Prolonged field operations deplete mobile device batteries, necessitating physical power bank distribution alongside relief kits.

# CHAPTER 6
# CONCLUSION AND FUTURE ENHANCEMENT

## 6.1 Conclusion

ReliefLink proves that a modern, event-driven web application can dramatically overcome the life-threatening inefficiencies of traditional disaster management in India. By replacing fragmented phone calls and chaotic social media forwards with an integrated, sub-second tactical coordination platform, disaster control authorities achieve comprehensive situational mastery. Stranded citizens gain immediate voice and geolocation visibility; accredited volunteer squads are dispatched with surgical spatial precision; emergency relief shelters maintain transparent occupancy metrics; and the automated SLA escalation daemon safeguards against neglected human distress.

### Achievements

- **Full-Stack Integration:** Delivered an integrated platform combining React 19, Leaflet GIS, Express, Socket.IO, and embedded PostgreSQL without external hosting friction.
- **Real-Time Dispatch:** Sub-200ms WebSocket event dissemination connects emergency control centres with hundreds of volunteer teams simultaneously.
- **Automated SLA Enforcement:** Engineered a resilient 60-second background daemon that eliminates stagnant distress beacons through automatic priority escalations.
- **Field-Ready Emergency UX:** Constructed a high-contrast dark visual theme optimized for outdoor glare and night rescue operations.

### Limitations

- **Requires terrestrial or mobile IP connectivity for initial distress transmission.**
- **Volunteer identity verification is currently managed administratively rather than via automated biometric Aadhaar authentication.**

### What I learned

This capstone project provided deep practical insights into full-stack architecture design, asynchronous WebSocket event synchronization, geospatial data visualization with Leaflet, embedded relational database transactions in PostgreSQL, and the vital responsibility of software engineering in humanitarian and life-safety applications.

## 6.2 Future Enhancements

The architectural roadmap for future iterations of ReliefLink includes:

- **Satellite & SMS Mesh Networking:** Integrate low-bandwidth SMS fallback gateways and LoRaWAN mesh transmitters for zero-cellular disaster zones.
- **AI-Powered Triage Voice Ingestion:** Deploy automatic speech recognition (ASR) capable of transcribing local regional languages (Tamil, Hindi, Telugu, Bengali) over emergency voice calls.
- **Autonomous Drone Imagery Feeds:** Connect live aerial video streams from survey drones to verify flood levels and road blockages directly on the tactical map.
- **Aadhaar / DigiLocker Volunteer Credentialing:** Enable instant digital identity and medical certification verification for spontaneous Good Samaritan volunteers.

**Table 6.1 Roadmap of future work**

| Phase | Planned Enhancement | Expected Operational Impact |
|---|---|---|
| 1. Network | Offline SMS & LoRaWAN mesh relay fallback | Continuous SOS reporting during complete cellular blackout |
| 2. Intelligence | AI voice distress transcription in 12 languages | Zero barrier for non-literate and elderly citizens |
| 3. Aerial GIS | Live drone video feed overlays on Leaflet map | Visual ground truth confirmation of inundated streets |
| 4. Identity | Aadhaar e-KYC integration for volunteer squads | Guaranteed field responder authenticity and rapid vetting |
| 5. Logistics | RFID & QR-code disaster ration tracking | Prevents relief diversion and guarantees equitable food distribution |

# REFERENCES / BIBLIOGRAPHY

[1] National Disaster Management Authority (NDMA), Government of India. "National Disaster Management Guidelines & Standard Operating Procedures." https://ndma.gov.in/

[2] World Meteorological Organization (WMO). "Guidelines on Multi-hazard Early Warning Systems and Emergency Communications." WMO-No. 1150.

[3] National Disaster Response Force (NDRF), India. "Standard Operating Procedures for Flood Rescue and Urban Relief Operations." https://ndrf.gov.in/

[4] OpenJS Foundation. "Express - Fast, unopinionated, minimalist web framework for Node.js." https://expressjs.com/

[5] The PostgreSQL Global Development Group. "PostgreSQL 16 Documentation: Relational Database Architecture and Indexing." https://www.postgresql.org/docs/

[6] ElectricSQL. "PGlite - Embedded PostgreSQL for WebAssembly and Node.js." https://pglite.dev/

[7] Meta Open Source. "React 19 Documentation: Concurrent Rendering and Server Actions." https://react.dev/

[8] Vladimir Agafonkin. "Leaflet: An open-source JavaScript library for mobile-friendly interactive maps." https://leafletjs.com/

[9] Socket.IO Authors. "Socket.IO: Bidirectional and low-latency communication for every platform." https://socket.io/docs/v4/

[10] Tailwind Labs. "Tailwind CSS: Rapidly build modern websites without leaving your HTML." https://tailwindcss.com/docs

[11] Source Code Repository. "ReliefLink - Real-Time Disaster Relief Coordination Platform." https://github.com/sanjay-k/ReliefLink.git

# APPENDICES

*(Source Code)*

The structure and excerpts below show the intended design of the implementation. Names of files, functions and fields may differ slightly in the final repository.

```
1. Project Structure
ReliefLink/
|-- server.ts                   // Express & Socket.IO server setup, Vite middleware
|-- server/
|   |-- db.ts                   // PostgreSQL (PGlite) client & schema migrations
|   |-- socket.ts               // Socket.IO event registration & room manager
|   |-- routes/
|   |   |-- incidents.ts        // REST APIs: /api/incidents, /api/sos
|   |   |-- volunteers.ts       // REST APIs: /api/volunteers, /api/dispatch
|   |   |-- shelters.ts         // REST APIs: /api/shelters
|   |   `-- reports.ts          // Incident audit trail CSV export streaming
|   `-- services/
|       `-- slaWorker.ts        // 60-second autonomous SLA escalation daemon
`-- src/
    |-- components/
    |   |-- TacticalMap.tsx     // Leaflet GIS interactive map component
    |   |-- SOSModal.tsx        // One-tap citizen SOS reporting modal
    |   |-- IncidentQueue.tsx   // Real-time verified distress incident cards
    |   |-- VolunteerPortal.tsx // Volunteer task force registry and dispatch
    |   `-- ShelterMonitor.tsx  // Relief camp bed & ration tracking
    `-- pages/
        `-- Dashboard.tsx       // Main tactical situation overview
```

```
2. Package Installation
npm install express socket.io @electric-sql/pglite cors dotenv zod leaflet
npm install -D typescript @types/node @types/express @types/leaflet vite tailwindcss
```

```
3. Environment Variables (.env)
PORT=3000
NODE_ENV=production
DATABASE_URL=memory://relieflink_pglite
SLA_EVALUATION_INTERVAL_MS=60000
MAP_TILE_PROVIDER=https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png
```

```
4. PostgreSQL Database Schema (server/db.ts)
CREATE TABLE IF NOT EXISTS incidents (
  id VARCHAR(64) PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  category VARCHAR(64) NOT NULL,
  priority VARCHAR(16) NOT NULL DEFAULT 'P2',
  status VARCHAR(32) NOT NULL DEFAULT 'Reported',
  latitude DOUBLE PRECISION NOT NULL,
  longitude DOUBLE PRECISION NOT NULL,
  address TEXT,
  casualties INTEGER DEFAULT 1,
  reporter_phone VARCHAR(32),
  escalated BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_incidents_status ON incidents(status);
CREATE INDEX IF NOT EXISTS idx_incidents_coords ON incidents(latitude, longitude);

CREATE TABLE IF NOT EXISTS volunteers (
  id VARCHAR(64) PRIMARY KEY,
  name VARCHAR(128) NOT NULL,
  phone VARCHAR(32) NOT NULL,
  skill_set TEXT NOT NULL,
  status VARCHAR(32) DEFAULT 'Available',
  assigned_incident_id VARCHAR(64) REFERENCES incidents(id)
);

CREATE TABLE IF NOT EXISTS shelters (
  id VARCHAR(64) PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  district VARCHAR(128) NOT NULL,
  capacity INTEGER NOT NULL,
  current_occupancy INTEGER DEFAULT 0,
  food_stock_days INTEGER DEFAULT 3,
  medical_kits INTEGER DEFAULT 50
);
```

```
5. Distress SOS Ingestion Controller (server/routes/incidents.ts)
router.post("/sos", async (req: Request, res: Response) => {
  const { category, latitude, longitude, phone, casualties, notes } = req.body;
  if (latitude < 8.0 || latitude > 37.5 || longitude < 68.0 || longitude > 97.5) {
    return res.status(400).json({ error: "Location outside operational boundaries" });
  }
  const id = `INC-${Date.now().toString().slice(-6)}`;
  const priority = (category === "Medical" || casualties >= 5) ? "P0" : "P1";
  const db = await getDb();
  await db.query(`
    INSERT INTO incidents (id, title, category, priority, status, latitude, longitude,
reporter_phone, casualties)
    VALUES ($1, $2, $3, $4, 'Reported', $5, $6, $7, $8)
  `, [id, `${category} Distress Request`, category, priority, latitude, longitude, phone,
casualties || 1]);

  getIo().emit("incident:created", { id, category, priority, latitude, longitude, casualties });
  return res.status(201).json({ success: true, trackingId: id, priority });
});
```

```
6. Real-Time Socket.IO Broadcast Handler (server/socket.ts)
export function initSocketServer(httpServer: any) {
  const io = new Server(httpServer, { cors: { origin: "*" } });
  io.on("connection", (socket) => {
    console.log(`[Socket.IO] Tactical operator connected: ${socket.id}`);
    socket.on("join:district", (district: string) => {
      socket.join(district);
      console.log(`[Socket.IO] Client ${socket.id} joined district room: ${district}`);
    });
    socket.on("disconnect", () => {
      console.log(`[Socket.IO] Operator disconnected: ${socket.id}`);
    });
  });
  return io;
}
```

```
7. Incident Audit CSV Export Route (server/routes/reports.ts)
router.get("/incidents.csv", async (req: Request, res: Response) => {
  const db = await getDb();
  const { rows } = await db.query(`
    SELECT id, title, category, priority, status, latitude, longitude, casualties, created_at
    FROM incidents ORDER BY created_at DESC
  `);
  const header =
"Incident_ID,Title,Category,Priority,Status,Latitude,Longitude,Casualties,Timestamp";
  const lines = [header].concat(
    rows.map(r => [
      r.id, `"${r.title}"`, r.category, r.priority, r.status,
      r.latitude, r.longitude, r.casualties, new Date(r.created_at).toISOString()
    ].join(","))
  );
  res.setHeader("Content-Type", "text/csv");
  res.setHeader("Content-Disposition", "attachment; filename=relieflink_incident_audit.csv");
  return res.send(lines.join("\n"));
});
```

```
8. REST API Request and Response Example
POST /api/sos HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{ "category": "Medical", "latitude": 13.0827, "longitude": 80.2707, "casualties": 3, "phone":
"9876543210" }

// Response: HTTP/1.1 201 Created
{ "success": true, "trackingId": "INC-819231", "priority": "P0", "assignedSLA": "15 mins" }
```